

R001 — Code Review: `hiddenSelect`

Multiple Shooting with Guess Propagation for Sparse Model Selection

Review prepared for Manu Jayadharan

10 September 2026

Abstract

Review of the Julia repository `hiddenSelect` — multiple shooting with guess propagation for discovering governing equations from noisy trajectory data — and of the accompanying theory manuscript `Multishooting_and_sparse_optimization` (`main.tex`). Covers what the theory claims, what the code computes, a headless re-execution of the numerical pipeline, and a ranked list of verified defects. The central algorithmic claim reproduces: sweeping the shooting-window size upward while carrying the previous optimum forward as the next initial guess drives the parameter error from $\|p_0 - p^*\| = 1.568$ at the seed to 0.156, whereas the same optimiser started from that seed at single-shooting granularity diverges and returns the blow-up penalty. Seven findings are reported; two silently corrupt published numbers — a dropped Julia line-continuation in the stiff/`DiffEq` loss branch that discards the entire w -residual (11–17% of the loss at the optimum), and the same defect in the Jacobian behind the stability/eigenvalue figures, which reduces a cubic vector field to its linear truncation and flips the sign of $\partial\dot{v}/\partial v$. Every quantitative claim below is gated: **14 of 14 gates pass** (`analysis/results/gates_summary.json`), including a cellwise check of the section 5 table and bug-presence gates for F1–F3 that are designed to *flip to failing* once those defects are fixed.

1 Scope, provenance and what was executed

Artefacts reviewed. The `hiddenSelect` repository (`~/git_repos/hiddenSelect`) at commit `8648cde` on branch `main`, whose working tree carried one whitespace-only modification to `fhn_model_selection_mod 8.jl`; and the theory archive `Multishooting_and_sparse_optimization.zip` (`main.tex`, 656 lines; Jayadharan, Hastewell and Mangan, 2025).

What was executed. Julia 1.11.3, with the committed `Project.toml` and `Manifest.toml`. The non-plotting stack (`DifferentialEquations 7.15`, `Optim 1.11`, `ForwardDiff 0.10.38` and `DataFrames 1.7`) precompiles and loads cleanly. Every experiment below loads the *verbatim source lines* of the repository files via `include_string` on a line range, so the functions exercised are the ones on disk. The `CairoMakie/GLMakie` stack could not be precompiled here — it requires rebuilding the REPL standard library, whose precompile-statement generator opens a pseudo-terminal (`/dev/ptmx`), which this sandbox denies (`Failed to open ptm`). That is an environment restriction, not a defect in the code; all plotting-free paths were exercised in full.

Self-containment. This folder regenerates itself. `deps/` holds a byte-identical, SHA-256-manifested snapshot of the eight Julia sources analysed, taken at commit `8648cde`; `analysis/common.jl` resolves `deps/` first and the analysis *never reads the live tree*. That matters specifically here: the analysis extracts source by *line range*, so an edit to a live file would otherwise shift those ranges silently. The dataset is not transcribed either — it is produced by executing frozen `mod 7` lines 60–107 verbatim (line 108 begins the plotting block). `REPRODUCE.md` pins the environment and the ordered commands; `analysis/verify_R001.py` writes `analysis/results/gates_summary.json`.

Scope — a newer commit exists upstream. This review measures local `HEAD = 8648cde`. A `git fetch after` the report was written showed `origin/main` carrying one further commit the local clone lacked: `57e430d`, “*Made all changes to make the loss function, problem setup and plotting compatible with generic N-dimensional ode. Not tested yet.*”, touching `mod 7`, `mod 8` and `plotting_window_size.jl`. No number here is wrong — `deps/` pins exactly what was measured — but three findings are no longer current upstream: **F2 is fixed in mod 8** (replaced by a matrix-norm accumulation, so it survives only in `mod 5`, `mod 6` and `mod 7`) and **both halves of F3 are fixed**. F1, F4 and F5 still stand at `origin/main`, and the version-lineage conclusion in §4 is superseded. `REPORT_ID.md` carries the finding-by-finding comparison. Re-reviewing `57e430d` is R002’s job, not a rewrite of R001.

2 Definitions (read before the results)

Everything the later sections use is defined here, including terms inherited from `main.tex` and from earlier script versions.

Model, data and estimation problem

The target system is FitzHugh–Nagumo (FHN), written as a general two-dimensional cubic polynomial vector field so that model *selection*, not merely parameter fitting, is the task:

$$\frac{d\mathbf{x}}{dt} = \mathbf{F}(\mathbf{x}; \mathbf{p}), \quad \frac{dx_i}{dt} = \sum_{j=1}^J p_{ij} \Theta_j(\mathbf{x}), \quad i = 1, \dots, n, \quad (1)$$

with $\mathbf{x} = (v, w)^\top$, $n = 2$, library $\Theta = \{1, v, w, v^2, vw, w^2, v^3, v^2w, vw^2, w^3\}$ ($J = 10$), and data $y_i = x(t_i) + \eta_i$, $\eta_i \sim \mathcal{N}(0, \sigma^2)$.

Methods / arms

Name	Full definition (not a citation)
<i>Single shooting</i> (SS)	One trajectory integrated from a single initial condition $x_0 \approx y_0$ across the whole record; cost is the summed squared residual at every data time. Equivalent to window size $N - 1$ (one window). Code: <code>forward_simulation_loss</code> , or <code>forward_simulation_loss_windows</code> with <code>window_size = N - 1</code> .
<i>Multiple shooting</i> (MS)	$[t_0, t_N]$ is partitioned into K windows $[\tau_{k-1}, \tau_k]$; each is integrated <i>from the datum at its left endpoint</i> $y_{\tau_{k-1}}$, not from a free decision variable. Function: <code>forward_simulation_loss_windows</code> . Read this: continuity across nodes is not enforced by constraint or penalty — unlike textbook multiple shooting the node states are <i>not</i> optimisation variables, as <code>main.tex</code> states.
<i>Guess propagation</i> (GP)	The outer loop. Window size is swept small (many nodes, near-convex) to large (few nodes, many local minima); the minimiser of each easier problem seeds the next. Code: <code>copyto!(x0, Optim.minimizer(optres))</code> inside for <code>window_size</code> in <code>window_size_range</code> .
<i>Best-guess propagation</i>	Variant in <code>mod 5/mod 8</code> : the guess is updated only on improvement (<code>if Optim.minimum(optres) < best_min</code>). Plain GP always overwrites.
<i>No propagation</i>	Control arm: every window size optimised from the <i>same</i> fixed seed (<code>copyto!(x0, x0_)</code>). This is the arm active in the committed <code>mod 8</code> .
<i>FS polish</i>	After the sweep, the best parameter vector is handed to a BFGS single-shooting optimisation (<code>optres2</code>) — the “better initial guess for a full ODE solver” step of <code>main.tex</code> .
<i>Data assimilation</i> (DA)	<code>data_assimilation_loss</code> : weak-constraint form in which the <i>whole discrete trajectory</i> X is an optimisation variable, data term weighted α , model-residual term weighted β . Present in every version but commented out of every driver — dead code.

Read this: two different parametrisations of the same sweep coexist in the repository and are easy to confuse. *Number of windows* K (used by `fhn_model_selection_mod 2.jl`, `Plotting_num_windows.jl`, and `figs/num_windows_plots/`): the record is cut into K equal chunks, so large $K =$ easy problem. *Window size* κ (used by `mod 5-mod 8`, `plotting_window_size.jl`, and `figs/window_size_plots/`): the number of *data intervals* spanned by one window, so large $\kappa =$ hard problem. The two sweeps therefore run in *opposite* directions; figures from the two families are not directly comparable.

Metrics and flags

Metric	Definition
$J_K(p)$ (<i>primary</i>)	Multiple-shooting cost, Eq. (2); returned as <code>data_loss/(2*length(D_1)) + sparse_loss/length(p_)</code> — squared residual sum over the total number of scalar observations, plus the mean smooth- ℓ_1 penalty over the 20 coefficients.
$\ p - p^*\ $	Euclidean distance between the fitted 20-vector and the true FHN coefficients. <i>Not</i> computed by the repository; added here as the identification-quality metric, since a low J_K with a wrong p is the failure mode of interest.
<i>Blow-up penalty</i>	If any state exceeds 10^3 or becomes NaN , the loss returns the flat constant 10^3 and records the offending (x, p) in <code>unstable_param</code> . Read this: this is a <i>plateau</i> , not a barrier: a value of exactly 1000.0 below means the optimiser never found a stable trajectory and the reported minimiser is just the seed.
<code>unstable_points</code>	List of $(x_{\text{window start}}, p)$ pairs that triggered the penalty; consumed by the <code>mod 6</code> stability analysis.
<code>smoothl1(x, α)</code>	$\alpha^{-1}[\log(1 + e^{-\alpha x}) + \log(1 + e^{\alpha x})]$ for $ \alpha x \leq 40$, else $ x $: a C^∞ surrogate for $ x $ with $\alpha = 500$. The sparsity-promoting term.

Symbols and frozen constants

Symbol	Meaning	Value here	Frozen in
δt	integrator step	$\Delta t/S = 0.1$	driver, S = 10
Δt	data spacing	1.0	downsample = 100 on a 0.01 grid
S	integrator steps per data interval	10	driver
κ	window size (data intervals per window)	swept 1...100	window_size_range
K	number of windows = $\lceil (N-1)/\kappa \rceil$	derived	—
N	number of data times	101	crop to $t \in [50, 150]$
γ_2	sparsity weight, forward-sim losses	5×10^{-2}	driver
γ_1	sparsity weight, DA loss	5×10^{-3}	driver (dead code)
α, β	DA data / model weights	1.0, 100.0	driver (dead code)
α_{ℓ_1}	sharpness of smoothl1	500	smoothl1 default
σ_{rel}	relative noise level	0.05	noise_sigma
N_p	number of library coefficients	$20 = 2 \times 10$	const Np
$L(p)$	Lipschitz constant of f in x	theory only	main.tex
$\tilde{L}$	Lipschitz constant of f in p	theory only	main.tex
$\mu, \lambda_{\min}$	strong-convexity / Hessian floor	theory only	main.tex

Read this: $\sigma_{\text{rel}} = 0.05$ is applied *per component relative to that component's own standard deviation*. Because w has roughly 40% of the amplitude of v , the absolute noise on w is about $2.5\times$ smaller, and the w -residual contributes only $\sim 12\%$ of $J_K(p^*)$. This matters for finding F2 below.

3 The theory in `main.tex`

The manuscript sets up the inverse problem (1) and introduces the flow map $\varphi_f(t; p, x_0)$ satisfying the semigroup property $\varphi_f(t_1 + t_2; p, x_0) = \varphi_f(t_2; p, \varphi_f(t_1; p, x_0))$. The multiple-shooting cost is

$$J_K(p) = \sum_{k=1}^K \sum_{t_i \in [\tau_{k-1}, \tau_k]} \left\| \varphi_f(t_i - \tau_{k-1}; p, y_{\tau_{k-1}}) - y_i \right\|^2, \quad (2)$$

reducing for $K = 1$ to the single-shooting cost J_1 . Four results are proved.

Lemma 1 (two-sided flow sensitivity). For Lipschitz f , $e^{-L(p)t} \|x_1 - x_2\| \leq \|\varphi_f(t; p, x_1) - \varphi_f(t; p, x_2)\| \leq e^{L(p)t} \|x_1 - x_2\|$.

Convergence of J_K to the noise-free cost. Splitting each residual into a flow-difference term and a noise term gives, with $\Delta T := \max_k (\tau_k - \tau_{k-1})$, $J_K(p) \leq N(e^{L(p)\Delta T} \|\eta\|_{\max} + \|\varphi_f(t_{i+1}; p, x_0) - x_{i+1}\| + \|\eta\|_{\max})^2$, so $J_K \rightarrow J^*$ as $\eta \rightarrow 0$. The single-shooting bound carries $e^{L(p)(t_i - t_0)}$ in place of $e^{L(p)\Delta T}$ — *this is the entire argument for multiple shooting*: the exponential amplification of the initial-condition error is capped at the window length rather than the record length. Under local strong convexity with modulus μ ,

$$\|p^* - p^{(K)}\| \leq \sqrt{\frac{2}{\mu} \left(N \|\eta\|_{\max}^2 (1 + e^{L(p^*)\Delta T})^2 - J_K(p^{(K)}) \right)}. \quad (3)$$

Lemma 2 (parameter sensitivity of the flow). $\|\varphi_f(t; p_1, x_0) - \varphi_f(t; p_2, x_0)\| \leq \frac{\tilde{L}}{L(p)} (e^{L(p)t} - 1) \|p_1 - p_2\|$.

Proposition 1 and the Theorem (node removal). For two nested partitions — $\{\tau_k\}$ and the coarser $\{\tau_k\}_{k \notin \mathcal{I}_R}$ obtained by deleting the nodes indexed by $\mathcal{I}_R$ — with $\Delta T_1 = \max_k (\tau_k - \tau_{k-1})$ and $\Delta T_2 = \max_k (\tau_k - \tau_k^-)$,

$$\|\hat{\mathcal{J}}_K(p) - J_K(p)\| \leq 2|\mathcal{I}_R|C_{\max} \left(e^{L(p)(\Delta T_1 + \Delta T_2)} \|\eta\|_{\max} + e^{L(p)\Delta T_1} \left[\frac{\tilde{L}}{L(p)} (e^{L(p)\Delta T_2} - 1) \|p - p^*\| + \|\eta\|_{\max} \right] \right), \quad (4)$$

and, under local strong convexity, $\|\hat{p}^{(K)} - p^{(K)}\|$ is bounded by $\frac{2}{\sqrt{\mu}}$ times the square root of a comparable expression. The practical reading, stated as a Note in the manuscript, is the design rule for guess propagation: *remove few nodes at a time, prefer nodes with small η_{τ_k} , and keep ΔT_2 small*. This is exactly the greedy sweep the code performs.

Gaps in the manuscript itself. (i) The flow map is defined with $p \in \mathbb{R}$ throughout — the manuscript carries its own reminder to make p multidimensional — yet every application has $p \in \mathbb{R}^{20}$. Relatedly, a cubic vector field is not globally Lipschitz, so $L(p)$ exists only on an invariant compact set; that should appear as a hypothesis. (ii) The bounds on J_K are one-sided; turning them into a bound on $\|p^* - p^{(K)}\|$ needs the Hessian/strong-convexity condition to hold *uniformly along the sweep*. That assumption does all the work and is never checked numerically — the contour and surface movies in `figs/` are the natural evidence for it and are not yet tied back to the theorem. (iii) The sparsity term $\gamma \sum_j \text{smoothl1}(p_j)$ that the code always uses appears nowhere in the analysis, so the code's $p^{(K)}$ is biased relative to the theorems'.

4 What the code actually does

Numerical core. `integrator.jl` is a clean, allocation-free fixed-step Tsitouras 5(4) integrator: typed tableau struct, preallocated `Tsit5Cache`, in-place and out-of-place step functions, fixed-step `integrate` driver. It is generic in `eltype`, which is why `ForwardDiff` differentiates straight through it. The strongest code in the repository.

Library and RHS. `helper_functions.jl` generates all multi-indices of total degree $\leq \text{deg}$ in `dim` variables and maps them to monomial strings; `mod 8` uses them in `odefun_poly!`, a dimension- and degree-agnostic polynomial RHS with a precomputed power table. *Verified:* with the re-ordered coefficient vector it reproduces the hard-coded `odefun` to 3.6×10^{-15} over 200 random states, so the refactor’s re-ordering of `fhn_p` is correct. But the monomial order changes from $(1, v, w, v^2, vw, w^2, v^3, v^2w, vw^2, w^3)$ to $(1, w, w^2, w^3, v, vw, vw^2, v^2, v^2w, v^3)$, and the bar-chart tick labels `trmstr` in `plotting_window_size.jl` still encode the *old* order — every coefficient plot drawn after the refactor lands will be mislabelled unless `trmstr` is regenerated from `multiindex_mapping`.

Loss and optimisation. `forward_simulation_loss_windows` has two branches: the *explicit* one (`stiff_solver_ = false`) marches the in-house Tsit5 with S substeps per data interval, resetting the state to the data at every window start (`if (i-1) % window_size_ == 0`); the *stiff* one builds an `ODEProblem` per window and calls a `DiffEq` solver (`Rosenbrock23`, `RadauIIA5`, ...) via `remake`. The explicit branch is what the driver optimises. Optimisers are `NelderMead` in the sweep and `BFGS` for the polish; *verified* that `ForwardDiff` differentiates the whole windowed loss (finite $\|\nabla J\| = 0.173$ at p^* , $\kappa = 5$).

Version lineage — which file is the latest working copy

File	Last commit	State
<code>fhn_model_selection.jl</code>	2025-01-31	Original SS + DA prototype.
<code>..._mod.jl</code> , <code>..._mod 2.jl</code>	2025-02-04	First windowed loss, parametrised by <i>number of windows</i> . <code>mod 2</code> carries an off-by-one: the residual after S substeps is compared against <code>D_1[i]</code> rather than <code>D_1[i+1]</code> .
<code>..._mod 3/4.jl</code>	2025-02-04/12	Contour-plot machinery added.
<code>..._mod 5.jl</code>	2025-02-12	Switch to <i>window size</i> ; plain and best-guess propagation; contour/surface movies. Generated <code>figs/window_size_plots/</code> .
<code>..._mod 6 ..._jl</code>	2025-03-31	Adds <code>unstable_param</code> collection and the eigenvalue/stability movies.
<code>..._mod 7.jl</code>	2025-04-04	Last version whose main pipeline runs. Lines 1–695 are coherent and were exercised end to end in this review. The trailing scratch block (lines ~697–881) does not run: it reassigns <code>const Np</code> , uses <code>multi_index_set</code> , <code>d</code> , <code>deg</code> and <code>fhn_p_2</code> before they are defined, and uses <code>@SVector</code> before using <code>StaticArrays</code> .
<code>..._mod 8.jl</code>	2025-04-07	Newest, but mid-refactor and broken — see F3/F4. Commit message: “started adding dimension agnostic code ... Need to extend the effect of changing <code>odefun</code> to the rest of the code”. Halts at line 97.

Answer to “which is the latest working copy”: `fhn_model_selection_mod 7.jl`, lines 1–695. `mod 8` is the newest file but is an unfinished refactor and cannot be run as-is; its new `odefun_poly!` is correct and worth keeping. Separately, note that `mod 8`’s driver has guess propagation *disabled* (`copyto!(x0, x0_)`, “Resetting the initial guess for each new window size”) and a three-point `window_size_range = [1, 50, 100]`, so even once it runs it reproduces the *control* arm, not the method.

Notebooks. The three `.ipynb` files are exploratory Lotka–Volterra side studies, not part of the FHN pipeline. `Untitled.ipynb` is Python/SciPy. `Untitled1.ipynb` and `backward_euler_scaling_jl.ipynb` declare a Julia 1.9.3 kernel; `IJulia` is not installed in this depot, so both were re-run as extracted scripts under 1.11.3. `Untitled1.ipynb` runs to completion; `backward_euler_scaling_jl.ipynb` fails on `Optim 1.11` at `Fminbox(SimulatedAnnealing())`:

```
MethodError: no method matching reset! (::SimulatedAnnealing, ::SimulatedAnnealingState, ::BarrierWrapper, ::Vector{Float64})
```

Box-constrained simulated annealing is no longer supported; use `NelderMead` inside `Fminbox`, or `SAMIN`, or drop the box. `Untitled1.ipynb` also illustrates the motivating pathology: with true $(\alpha, \beta, \gamma, \delta) = (1, 0.5, 0.8, 0.3)$ the single-shooting fit converges to $(\approx 4.0, 2.9, 7.1, 2.9)$ — a confident fit to the wrong model.

5 Reproduction results

All runs use the committed data recipe: FHN integrated at $\delta t = 0.01$ to $T = 156$, downsampled by 100, cropped to $t \in [50, 150]$ ($N = 101$ points, $\Delta t = 1$), 5% per-component relative Gaussian noise, `Random.seed!(1287436679)`. Optimiser `NelderMead`, 2500 iterations, $\gamma_2 = 5 \times 10^{-2}$, $S = 10$, seed `Random.seed!(1)` giving $\|p_0 - p^*\| = 1.568$.

κ	Guess propagation		No propagation (control)	
	J_K	$\ p - p^*\ $	J_K	$\ p - p^*\ $
1	0.01338	1.1347	0.01338	1.1347
2	0.01360	0.3967	0.01608	1.0868
5	0.01316	0.1563	0.04694	1.8765
10	0.01055	0.1851	1000.0	1.5679
25	0.00991	0.2414	1000.0	1.5679
50	0.00977	0.2717	1000.0	1.5679
100	0.00960	0.2835	1000.0	1.5679

$\kappa = 100$ is single shooting over the whole record. The control arm returns the blow-up plateau 1000.0 for every $\kappa \geq 10$, and its reported minimiser is unchanged from the seed ($\|p - p^*\| = 1.5679$), i.e. the optimiser never found a stable trajectory. With propagation, the same optimiser reaches single shooting with $J_K = 0.0096$ and $\|p - p^*\| = 0.28$. **This is the paper’s claim, and it reproduces cleanly.**

Two further observations. J_K decreases monotonically in κ along the propagated arm, consistent with the theory: larger windows mean fewer noisy restarts, so the $e^{L\Delta T}\|\eta\|$ term is paid at fewer nodes. Less comfortably, $\|p - p^*\|$ is *not* monotone — it bottoms out at $\kappa = 5$ (0.156) and then *degrades* to 0.28 by $\kappa = 100$ while the cost keeps improving. The last sweep steps buy cost at the expense of identification. This is a real result, and exactly what the $\|p^* - p^{(K)}\|$ bounds of §3 should explain: intermediate κ balances the noise term against the conditioning term. It argues for reporting $\|p - p^*\|$ (or, on real data, a held-out forecast error) alongside J_K and for choosing the stopping point on it rather than running to $\kappa = N - 1$ by default. Sanity checks at the true parameters give $J_K(p^*) \in [0.0050, 0.0066]$ across $\kappa \in \{1, 5, 25, 100\}$, consistent with the noise floor, while random p draws (8 per κ , gate G12) give $J_K \in [0.0721, 1000]$ — every one of them above every $J_K(p^*)$, so the cost does discriminate.

6 Findings

Ordered by severity. Each was confirmed by execution, not by reading alone.

F1 — Stability/eigenvalue figures use a linear truncation of the vector field. (mod 6, `odefun_new`, lines 980–993; silent wrong numbers.) `odefun_new` writes each component as `F[1] = p[1] + p[2]*y[1] + p[3]*y[2]` followed by continuation lines beginning with `+ p[4]*y[1]^2 ...`. In Julia the first line is a *complete* statement; the following lines parse as separate, discarded expressions. Every quadratic and cubic term is dropped. Verified at $u = (1.5, 0.4)$, $p = p_{\text{FHN}}^*$: `odefun_new` = (1.600, 0.1504), correct RHS = (0.475, 0.1504), linear truncation = (1.600, 0.1504) — an exact match to the truncation. The Jacobian consumed by `get_eigens` is $\begin{pmatrix} 1.0 & -1.0 \\ 0.08 & -0.064 \end{pmatrix}$ instead of $\begin{pmatrix} -1.25 & -1.0 \\ 0.08 & -0.064 \end{pmatrix}$; the (1,1) entry has the *wrong sign*, which for a stability plot is the worst possible error. All movies under `figs/window_size_plots/stability/` are affected and should be regenerated. *Fix*: end each continued line with a trailing `+`, or wrap the RHS in parentheses — better still, delete `odefun_new` and differentiate the existing `odefun/odefun_poly!` through a one-line closure.

F2 — The stiff/DiffEq loss branch silently discards the entire w -residual. (mod 5:299–302, mod 6:317–320, mod 7:355–358, mod 8:409–412; silent wrong objective.) Same parsing trap: `data_loss += dot(...v ...)` terminates the statement, and the following `+ dot(...w ...)` is evaluated and thrown away. Quantified by running the file’s own function against a patched copy with the continuation repaired, at $p = p^*$:

κ	as written	w -term restored	fraction of the loss missing
1	0.0044519	0.0050288	11.5%
10	0.0032216	0.0038801	17.0%
25	0.0057854	0.0065134	11.2%
100	0.0043642	0.0049514	11.9%

The repaired stiff branch agrees with the explicit branch to all printed digits (0.0050288, 0.0038801, 0.0065134, 0.0049514), which is a good cross-validation of the two integrators once the bug is removed. The missing fraction is only $\sim 12\%$ at the optimum because the w noise is small; away from the optimum, where the w trajectory can be badly wrong, the omitted term is much larger, so the stiff branch optimises an objective that is *blind to w* . The explicit branch, which accumulates scalar `abs2` terms, is unaffected — so the main results are safe, but every contour/surface movie produced with `stiff_solver=true` (Rosenbrock23, RadauIIA5, adaptive-Tsit5 variants under `figs/window_size_plots/contour_plots/` and `surface_plots/`) shows the wrong landscape.

F3 — mod 8 does not run. (Two independent errors.) (a) Line 97: `prob_fhn = ODEProblem(..., fhn_u0, ...)` — the variable is named `fhn_y0`; `fhn_u0` was defined only in mod 7’s scratch block and was not carried over. Confirmed: `UndefVarError: fhn_u0`. (b) Line 260: `d, N = size(data)` where `data` is the flattened $2N$ -vector, so `size` returns a 1-tuple. Confirmed: `BoundsError: attempt to access Tuple{Int64} at index [2]`. In the same function `D_1/D_2` are commented out but still referenced throughout the body, so `fs_loss` — and therefore the final BFGS polish — cannot work either. (c)

Gating this surfaced a third defect not in the first draft: that `size(data)` reads the *global data*, silently ignoring the function’s own `data_` argument (gate G7). The signature is therefore a lie — passing a different dataset has no effect — and the same shadowing should be checked in `forward_simulation_loss_windows` when the refactor lands.

F4 — The window-sweep “gradient” differentiates the wrong function. (mod 5:530, mod 7, mod 8:573.) `grad_fs_loss_window!(g,x)` calls `ForwardDiff.gradient(fs_loss, x)`, not `fs_loss_window`. It is currently harmless only because the sweep uses `NelderMead`, which ignores the supplied gradient. The moment anyone switches the commented-out `BFGS` line back on — which the code invites — the sweep will descend on the single-shooting loss while reporting the windowed one. Given that `ForwardDiff` through the windowed loss was verified to work here, this is a one-word fix with a real payoff in speed over `NelderMead`.

F5 — Stale call signature in `animate_contourplots!`. (`plotting_window_size.jl:378–382`.) It calls `loss_function_window(x0, p, odefun_, ...; silent=true)`, but since mod 6 the signature is `forward_simulation_loss_windows(x0_, unstable_param, p_, ...; silent=...)` — the `unstable_param` argument was inserted and the keyword renamed. The helper will throw if called; the drivers work around it by inlining a private `record(...)` block instead, which is why this has gone unnoticed.

F6 — Thread-safety of the outer loop. `@threads for i in 1:num_runs` writes into the shared `x0`, `results_inner` and `unstable_points_dict`. With `num_runs = 1` this is benign, but the multi-seed study the loop is clearly built for will race: `Dict` is not thread-safe, and guess propagation through a shared `x0` is not even well-defined across threads. Make `x0`, `results_inner` and `unstable_points_dict` thread-local and reduce afterwards.

F7 — Smaller items. (i) mod 2’s `forward_simulation_loss_window` compares the state after S substeps to `D_1[i]` instead of `D_1[i+1]` — a one-interval misalignment, fixed by mod 5, but it means the earliest `figs/num_windows_plots/` figures were produced with a shifted residual. (ii) `partition_view` with `chunk_size = div(length(arr), num_windows)` emits an extra short trailing window when the length is not divisible, so the effective K silently exceeds `num_windows`. (iii) `forward_simulation_loss_windows` keyword defaults are written `abs_tol=abstol=1e-8`, which assigns a global named `abstol` as a side effect of evaluating the default; harmless but accidental. (iv) The blow-up penalty is a flat 10^3 with no gradient, so the optimiser gets no direction out of the unstable region; a penalty that grows with $\|x\|$ or with the index at which the blow-up occurred would guide it back. (v) `plotSolution!` references the global `optres` inside its `DA=true` branch, and `ax31` inside its `FS` branch even when `GFreeFS=false`; both will throw if those branches are ever used. (vi) `Statistics` and `Random` are used but absent from `Project.toml`.

7 Gates

Every quantitative claim in this report is asserted by `analysis/verify_R001.py`, which reads only this folder’s own `analysis/results/` and writes `gates_summary.json`. Current status: **14 of 14 pass**.

Gate		What it asserts
G1_deps_frozen_matches_manifest	pass	All 8 frozen sources hash to <code>deps/MANIFEST.md</code> .
G2_odefun_poly_equals_odefun	pass	$\max \text{odefun} - \text{odefun_poly} = 7.1 \times 10^{-15}$ over 200 random states (tol 10^{-12}).
G3_F1_bug_present_..._linear_truncation	pass	<code>odefun_new</code> equals the linear truncation and differs from the true RHS.
G4_F1_bug_present_jacobian_j11_sign_flipped	pass	$J_{11} = +1.0$ from <code>odefun_new</code> vs -1.25 true; off-diagonals agree.
G5_F2_bug_present_stiff_drops_w_residual	pass	Missing fraction in $[10\%, 20\%]$ at every κ tested.
G6_repaired_stiff_matches_explicit	pass	$\max \text{repaired} - \text{explicit} = 3.0 \times 10^{-10}$ (tol 10^{-8}).
G7_F3_bug_present_mod8_runtime_errors	pass	<code>UndefVarError</code> , <code>BoundsError</code> , and the global-data shadowing.
G8_section5_table_reproduces	pass	All 14 cells of the §5 table match the printed values to 5×10^{-5} .
G9_control_arm_is_plateau_not_a_fit	pass	Every control row with $\kappa \geq 10$ has J_K exactly 1000.0 and $\ p - p^*\ $ equal to the seed.
G10_propagation_better_cellwise	pass	Strictly better at $\kappa \in \{2, 5, 10, 25, 50, 100\}$; tied at $\kappa = 1$, where a tie is required.
G11_p_err_non_monotone_in_window_size	pass	$\ p - p^*\ $ minimised at an interior $\kappa = 5$ and rising to $\kappa = 100$ while J_K falls.
G12_cost_discriminates	pass	Every $J_K(p^*)$ below every random- p draw (8 per κ).
G13_forwarddiff_through_windowed_loss	pass	$\ \nabla J\ = 0.17344$, all entries finite.
G14_completeness_hashes	pass	7 result files present; SHA-256 ledger written.

Gate polarity — read before acting on a failure. The five gates whose names contain `bug_present` assert that a *defect is still there* in the frozen `deps/` snapshot. They pass *because* F1, F2 and F3 are unfixed. When those are repaired in the live tree and `deps/` is re-frozen, **G3, G4, G5 and G7 should flip to failing** — that flip is the evidence the fix landed, not a regression. G6 is the complement and must keep passing throughout. Every other gate is an ordinary correctness gate whose failure means something broke.

What is *not* gated. The theory review in §3 and the version-lineage judgements in §4 are readings, not measurements. The claim that `mod 7` lines 1–695 form the last coherent pipeline is supported by execution of its loss functions and data block (G2, G6, G8, G13) but is not itself a gate — the file’s plotting calls were never executed here, for the sandbox reason in §1. F5 (stale signature in `animate_contourplots!`) and F6 (thread-safety) are read from the frozen source and argued, not asserted: F5 would need the Makie stack, and F6 is a race whose absence a single run cannot demonstrate.

8 Recommendations

1. **Fix F1 and F2 first and regenerate the affected figures** — they are the only findings that change numbers already in circulation. The regression test that catches F2 now exists: gate `G6_repaired_stiff_matches_explicit` asserts the two branches agree at p^* to 10^{-8} , and `G5_F2_bug_present_stiff_drops_w_residual` will flip to failing the moment you repair the continuation. Re-freeze `deps/` after the fix so the flip is recorded.
2. **Finish the `mod 8` refactor from `mod 7`**, keeping `odefun_poly!` (verified correct) and repairing F3; regenerate `trmstr` from `multiindex_mapping` so the coefficient bar charts match the new monomial order. Re-enable guess propagation in the driver.
3. **Collapse the eight `mod N` scripts into one module plus thin drivers.** They share $\sim 80\%$ of their text; every fix has had to be applied n times, and F2 shows it was not. A `src/HiddenSelect.jl` plus per-experiment drivers makes the lineage question of §4 disappear.
4. **Report $\|p - p^*\|$, not just J_K** , and use it to choose where to stop the sweep. The non-monotonicity in §5 is a result, not an artefact, and it connects directly to the $\|p^* - p^{(K)}\|$ bounds.
5. **Switch the sweep to BFGS** once F4 is fixed: AD through the windowed loss already works, and `NelderMead` in 22 dimensions is the main reason the sweep needs 2500 iterations per step.
6. **Close the loop with the theory.** The manuscript’s node-removal Note prescribes removing few nodes at a time and preferring low-noise nodes; the code removes nodes by a fixed stride in κ and ignores η entirely. An arm implementing the prescribed greedy rule would be a direct numerical test of Proposition 1.
7. **Environment.** Pin the notebooks to Julia 1.11 (or record the 1.9.3 manifest) — `backward_euler_scaling.jl.ipynb` is already broken by the `Optim` API change. To exercise the `CairoMakie` paths in a future session the sandbox needs write access to `~/.julia` and to `/dev/ptmx`.